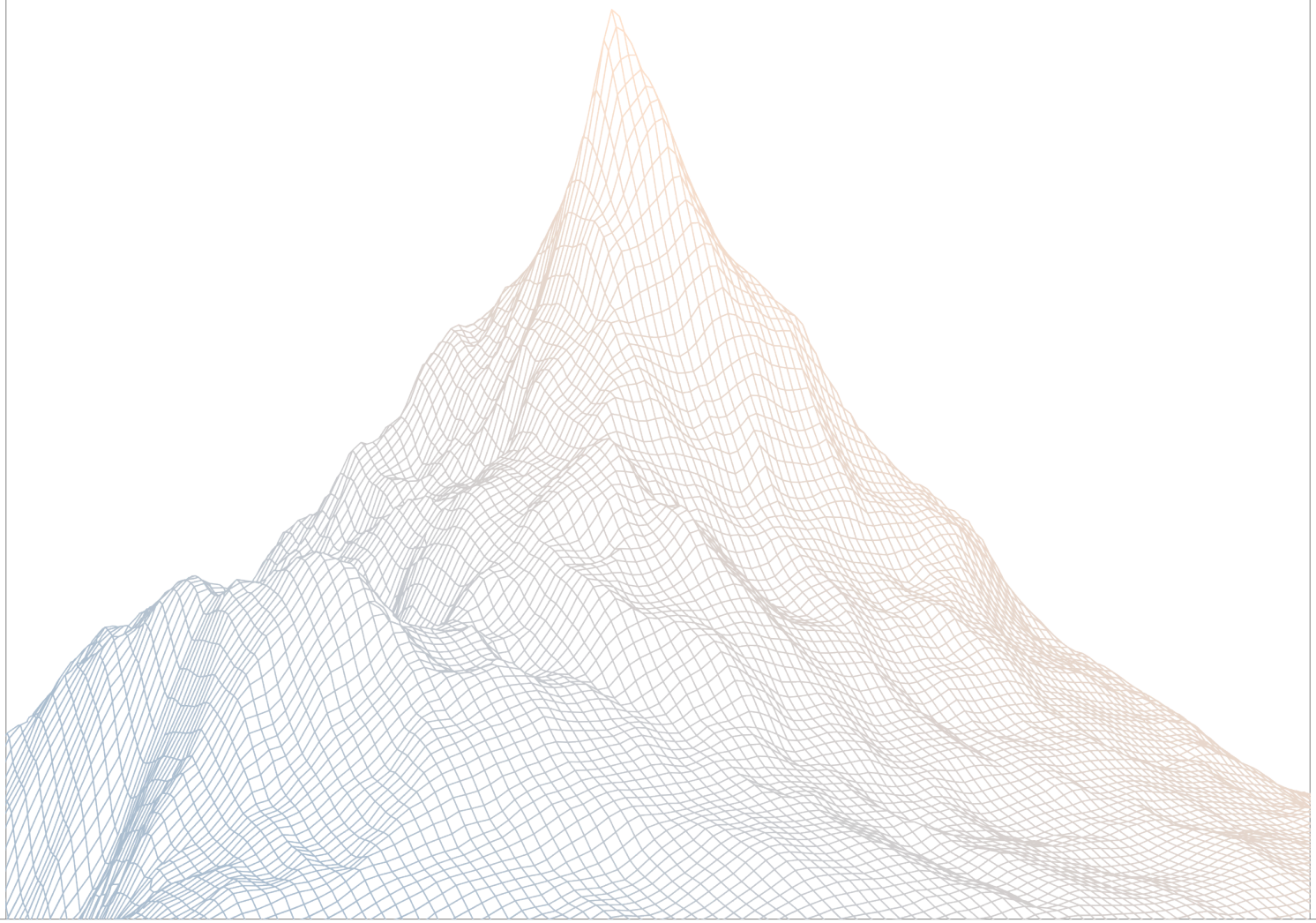
Zenith

Meteora

Smart Contract Security Assessment

VERSION 1.1



AUDIT DATES: February 23rd to February 26th, 2026

AUDITED BY: J4X

Contents

1 Introduction 2

1.1 About Zenith 3

1.2 Disclaimer 3

1.3 Risk Classification 3

2 Executive Summary 3

2.1 About Meteora 4

2.2 Scope 4

2.3 Audit Timeline 5

2.4 Issues Found 5

3 Findings Summary 5

4 Findings 6

4.1 Low Risk 7

4.2 Informational 19

1

Introduction

1.1 About Zenith

Zenith assembles auditors with proven track records: finding critical vulnerabilities in public audit competitions.

Our audits are carried out by a curated team of the industry's top-performing security researchers, selected for your specific codebase, security needs, and budget.

Learn more about us at https://zenith.security.

1.2 Disclaimer

This report reflects an analysis conducted within a defined scope and time frame, based on provided materials and documentation. It does not encompass all possible vulnerabilities and should not be considered exhaustive.

The review and accompanying report are presented on an "as-is" and "as-available" basis, without any express or implied warranties.

Furthermore, this report neither endorses any specific project or team nor assures the complete security of the project.

1.3 Risk Classification

SEVERITY LEVEL	IMPACT: HIGH	IMPACT: MEDIUM	IMPACT: LOW
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

2

Executive Summary

2.1 About Meteora

Our mission is to build the most secure, sustainable and composable liquidity layer for all of Solana and DeFi.

By using Meteora's DLMM and Dynamic AMM Pools, liquidity providers can earn the best fees and yield on their capital.

This would help transform Solana into the ultimate trading hub for mainstream users in crypto by driving sustainable, long-term liquidity to the platform. Join us at Meteora to shape Solana's future as the go-to destination for all crypto participants.

2.2 Scope

The engagement involved a review of the following targets:

Target	damm-v2
Repository	https://github.com/MeteoraAg/damm-v2
Commit Hash	`8168ac6e94bfb1940488593d14014f0c30d34aa7`
Files	`Changes in PR-187`

Target	Mitigation Review
Repository	https://github.com/MeteoraAg/damm-v2
Commit Hash	`6e9aee4e549bd792c8f5b82b88be04459e644f3c`
Files	`Changes in PR-187`

2.3 Audit Timeline

February 23, 2026	Audit start
February 26, 2026	Audit end
March 2, 2026	Report published

2.4 Issues Found

SEVERITY	COUNT
Critical Risk	0
High Risk	0
Medium Risk	0
Low Risk	4
Informational	2
Total Issues	**6**

3

Findings Summary

ID	Description	Status
L-1	All sdk quote function will return a sqrt price of 0 for compounding pools	Acknowledged
L-2	Operator can take part of protocol fee on fee zaps	Acknowledged
L-3	get_max_amount_in is off by one	Resolved
L-4	Inconsistent price behavior between compound and concentrated pool	Resolved
I-1	get_next_sqrt_price_from_output is documented incorrectly	Resolved
I-2	Inconsisten behavior in case of 0 swaps	Resolved

4

Findings

4.1 Low Risk

A total of 4 low risk findings were identified.

[L-1] All sdk quote function will return a sqrt price of 0 for compounding pools

SEVERITY: Low	IMPACT: Low
STATUS: Acknowledged	LIKELIHOOD: High

Target

- rust-sdk/src/quote_exact_in.rs#L32-L37
- rust-sdk/src/quote_exact_out.rs#L30-L37
- rust-sdk/src/quote_partial_fill_in.rs#L32-L40
- programs/cp-amm/src/liquidity_handler/compounding_liquidity.rs#L117-L144
- programs/cp-amm/src/instructions/swap/ix_p_swap.rs#L177-L180

Description:

The Rust SDK quote modules are used to estimate swap results off-chain. The expected behavior is that quote responses return a valid `next_sqrt_price` for the quoted swap. All three quote paths (`exact_in`, `exact_out`, and `partial_fill_in`) call pool swap-result functions and return the result directly.

```
Ok(pool.get_swap_result_from_exact_input(
    actual_amount_in,
    &fee_mode,
    trade_direction,
    current_point,
)?)
```

```
let swap_result = pool.get_swap_result_from_exact_output(
    actual_amount_out,
    &fee_mode,
    trade_direction,
    current_point,
)?;
```

```
Ok(swap_result)
```

```
let swap_result = pool.get_swap_result_from_partial_input(
    actual_amount_in,
    &fee_mode,
    trade_direction,
    current_point,
)?;

Ok(swap_result)
```

The issue appears in the compounding pool path. In this case, the pool liquidity functions return `next_sqrt_price` as `0`, and that value is returned by all quote paths above.

```
fn calculate_a_to_b_from_amount_out(&self, amount_out: u64) →
    Result<SwapAmountFromOutput> {
    let input_amount = safe_mul_div_cast_u64(
        self.token_a_amount,
        amount_out,
        self.token_b_amount.safe_sub(amount_out)?,
        Rounding::Up,
    )?;
    Ok(SwapAmountFromOutput {
        input_amount,
        next_sqrt_price: 0, // dont need to care for next sqrt price now
    })
}
```

```
fn calculate_b_to_a_from_amount_out(&self, amount_out: u64) →
    Result<SwapAmountFromOutput> {
    let input_amount = safe_mul_div_cast_u64(
        self.token_b_amount,
        amount_out,
        self.token_a_amount.safe_sub(amount_out)?,
        Rounding::Up,
    )?;
    Ok(SwapAmountFromOutput {
        input_amount,
        next_sqrt_price: 0, // dont need to care for next sqrt price now
    })
}
```

This happens because `0` is used as a placeholder in the compounding liquidity math, and

the SDK quote paths do not replace it. The execution path does replace it after applying the swap result.

```
pool.apply_swap_result(&swap_result, &fee_mode, trade_direction,
    current_timestamp)?;

// re-update next_sqrt_price for compounding pool
swap_result.next_sqrt_price = pool.sqrt_price;
```

As a result, the quote functions return an incorrect `next_sqrt_price` for compounding pools.

Recommendations:

We recommend normalizing `next_sqrt_price` in the SDK quote path for compounding pools before returning `SwapResult2`. The returned quote should use the same final price value that execution exposes (`pool.sqrt_price`). This can be achieved by calculating the price post swap, or adapting the pool functiosn to return the correct price.

Meteora: Acknowledged.

[L-2] Operator can take part of protocol fee on fee zaps

SEVERITY: Low	IMPACT: Low
STATUS: Acknowledged	LIKELIHOOD: Low

Target

- ix_zap_protocol_fee.rs

Description:

`zap_protocol_fee` tries to ensure claimed protocol fees are zapped to treasury-compatible destinations by validating the following zap-out instruction through `protocol-zap` introspection.

```
// cp-amm/programs/cp-amm/src/instructions/operator/ix_zap_protocol_fee.rs
validate_zap_out_to_treasury(
    amount,
    &crate::ID.to_bytes(),
    &ctx.accounts.receiver_token.key().to_bytes(),
    &ctx.accounts.receiver_token.try_borrow_data()?,
    &ctx.accounts.sysvar_instructions.try_borrow_data()?,
    &TREASURY::ID.to_bytes(),
    &treasury_paired_destination_token_address.to_bytes(),
)?;
```

The dependency check verifies:

- the next instruction is `zap::zap_out`,
- source token account matches the claimer account,
- destination account is treasury SOL/USDC ATA or paired treasury token ATA.

However, for DAMM v2 routes, `protocol-zap` does not validate who is the referrer:

```
// protocol-zap/src/processors/damm_v2_zap.rs
impl ZapInfoProcessor for ZapDammV2InfoProcessor {
    fn validate_payload(&self, _payload: &[u8]) -> Result<(),
    ProtozolZapError> {
        Ok(())
    }
```

```
}
```

In `cp-amm` swap execution, referral mode is toggled by whether `referral_token_account` ≠ `cp_amm::ID` sentinel, and then referral fees are transferred out:

```
// cp-amm/programs/cp-amm/src/instructions/swap/ix_p_swap.rs
let has_referral = referral_token_account.key().ne(crate::ID.as_array());
let fee_mode = FeeMode::get_fee_mode(collect_fee_mode, trade_direction,
    has_referral);
// ...
if has_referral {
    p_transfer_from_pool(
        pool_authority,
        token_a_mint_or_b_mint,
        token_a_vault_or_b_vault,
        referral_token_account,
        token_program,
        swap_result.referral_fee,
    )?;
}
```

The fee split logic confirms referral value is taken from protocol fee:

```
// cp-amm/programs/cp-amm/src/state/fee.rs
let referral_fee = if has_referral {
    safe_mul_div_cast_u64(protocol_fee, self.referral_fee_percent.into(),
    100, Rounding::Down)?
} else {
    0
};
let protocol_fee = protocol_fee.safe_sub(referral_fee)?;
```

Exploit path

1. Operator calls `zap_protocol_fee` and receives claimed fee amount into their temporary token account.
2. In the next `zap::zap_out`, operator chooses DAMM v2 swap payload/accounts that set a `referral_token_account` they control.
3. Swap executes and transfers `swap_result.referral_fee` to operator referral account.
4. Treasury receives output tokens, but net value is reduced by referral extraction.

Recommendations:

In `protocol-zap` validation for DAMM v2 zap-out, enforce no-referral route semantics for protocol-fee integrations:require `referral_token_account == cp_amm::ID`

Meteora: Acknowledged.

[L-3] get_max_amount_in is off by one

SEVERITY: Low	IMPACT: Low
STATUS: Resolved	LIKELIHOOD: High

Target

- programs/cp-amm/src/liquidity_handler/concentrated_liquidity.rs#L264-L285
- programs/cp-amm/src/liquidity_handler/concentrated_liquidity.rs#L120-L169
- programs/cp-amm/src/liquidity_handler/concentrated_liquidity.rs#L311-L369
- programs/cp-amm/src/math/u128x128_math.rs#L83-L100

Description:

The concentrated liquidity module computes swap amounts and enforces price-range boundaries for exact-in and exact-out paths. For exact-in boundary handling, the implementation is expected to use a rounded-up maximum input so the amount needed to fully reach the edge price is represented correctly.

```
fn calculate_a_to_b_from_partial_amount_in(
    &self,
    amount_in: u64,
) -> Result<SwapAmountFromInput> {
    let max_amount_in: ruint::Uint<256, 4>
    = get_delta_amount_a_unsigned_unchecked(
        self.sqrt_min_price,
        self.sqrt_price,
        self.liquidity,
        Rounding::Up,
    )?;
    // ...
}
```

```
fn calculate_b_to_a_from_partial_amount_in(
    &self,
    amount_in: u64,
) -> Result<SwapAmountFromInput> {
    let max_amount_in = get_delta_amount_b_unsigned_unchecked(
        self.sqrt_price,
```

```
        self.sqrt_max_price,
        self.liquidity,
        Rounding::Up,
    )?;
    // ...
}
```

In the helper `get_max_amount_in`, the same value is computed with `Rounding::Down` for both directions. This is lower than the actual rounded-up boundary when division has a remainder, so the returned value can be off by one.

```
#[cfg(test)]
fn get_max_amount_in(&self, trade_direction: TradeDirection) -> Result<u64>
    {
    let amount = match trade_direction {
        TradeDirection::AtoB => get_delta_amount_a_unsigned_unchecked(
            self.sqrt_min_price,
            self.sqrt_price,
            self.liquidity,
            Rounding::Down,
        )?,
        TradeDirection::BtoA => get_delta_amount_b_unsigned_unchecked(
            self.sqrt_price,
            self.sqrt_max_price,
            self.liquidity,
            Rounding::Down,
        )?,
    };
    // ...
}
```

This happens because the helper uses floor rounding while the boundary logic in the execution path uses ceiling rounding. As a result the helper returns an incorrect maximum.

Recommendations:

We recommend changing `get_max_amount_in` to use `Rounding::Up` so it matches the exact-in boundary logic used by concentrated liquidity swaps.

```
#[cfg(test)]
fn get_max_amount_in(&self, trade_direction: TradeDirection) -> Result<u64>
    {
    let amount = match trade_direction {
        TradeDirection::AtoB => get_delta_amount_a_unsigned_unchecked(
```

```
            self.sqrt_min_price,
            self.sqrt_price,
            self.liquidity,
            Rounding::Up,
        )?,
        TradeDirection::BtoA => get_delta_amount_b_unsigned_unchecked(
            self.sqrt_price,
            self.sqrt_max_price,
            self.liquidity,
            Rounding::Up,
        )?,
    };
    // ...
}
```

Meteora: Resolved with PR-195.

Zenith: Verified.

[L-4] Inconsistent price behavior between compound and concentrated pool

SEVERITY: Low	IMPACT: Low
STATUS: Resolved	LIKELIHOOD: High

Target

- programs/cp-amm/src/state/pool.rs#L851-L853
- programs/cp-amm/src/liquidity_handler/concentrated_liquidity.rs#L372-L408
- programs/cp-amm/src/liquidity_handler/compounding_liquidity.rs#L151-L167
- programs/cp-amm/src/state/fee.rs#L327-L339

Description:

The pool updates `sqrt_price` after each swap, then uses that stored value to update dynamic-fee volatility. This should behave in the same way for both pool types.

```
let liquidity_handler = self.get_liquidity_handler()?;
let next_sqrt_price =
    liquidity_handler.get_next_sqrt_price(next_sqrt_price)?;
self.sqrt_price = next_sqrt_price;
```

In concentrated mode, the next price comes from direction-aware swap math (`get_next_sqrt_price_from_input` / `get_next_sqrt_price_from_output`) with explicit rounding choices in those functions. In case of amount_in the price will round contrary to the direction of the swap, in case of amount_out it will round in the same direction.

```
pub fn get_next_sqrt_price_from_input(
    sqrt_price: u128,
    liquidity: u128,
    amount_in: u64,
    a_for_b: bool,
) -> Result<u128> {
    if a_for_b {
        get_next_sqrt_price_from_amount_in_a_rounding_up(sqrt_price,
    liquidity, amount_in)
    } else {
        get_next_sqrt_price_from_amount_in_b_rounding_down(sqrt_price,
```

```
    liquidity, amount_in)
    }
}

pub fn get_next_sqrt_price_from_output(
    sqrt_price: u128,
    liquidity: u128,
    amount_out: u64,
    a_for_b: bool,
) → Result<u128> {
    if a_for_b {
        get_next_sqrt_price_from_amount_out_b_rounding_down(sqrt_price,
    liquidity, amount_out)
    } else {
        get_next_sqrt_price_from_amount_out_a_rounding_up(sqrt_price,
    liquidity, amount_out)
    }
}
```

In compounding mode, `get_next_sqrt_price` ignores the swap-calculated value and always recomputes price from reserves via integer division and integer square root. This path always floors due to integer math.

```
fn get_next_sqrt_price(&self, _next_sqrt_price: u128) → Result<u128> {
    get_sqrt_price_from_amounts(self.token_a_amount, self.token_b_amount)
}

fn get_sqrt_price_from_amounts(token_a_amount: u64, token_b_amount: u64) →
    Result<u128> {
    let token_b_amount = U256::from(token_b_amount).safe_shl(128)?;
    let price = token_b_amount.safe_div(U256::from(token_a_amount))?;
    let sqrt_price = sqrt_u256(price).ok_or_else(||
    PoolError::MathOverflow)?;
    Ok(sqrt_price.try_into().map_err(|_| PoolError::TypeCastFailed)?)
}
```

This happens because concentrated uses swap-path rounding to produce `next_sqrt_price`, while compounding replaces it with reserve-derived floor quantization before the dynamic-fee update reads it. As a result, compounding and concentrated pools can update dynamic-fee volatility differently for equivalent swap price moves.

```
pub fn update_volatility_accumulator(&mut self, sqrt_price: u128) →
    Result<()> {
    let delta_price =
        Self::get_delta_bin_id(self.bin_step_u128, sqrt_price,
```

```
    self.sqrt_price_reference)?;
    let volatility_accumulator = self
        .volatility_reference
        .safe_add(delta_price.safe_mul(MAX_BASIS_POINT.into())?)?;
    self.volatility_accumulator = std::cmp::min(
        volatility_accumulator,
        self.max_volatility_accumulator.into(),
    );
    Ok(())
}
```

Additionally the market cap based base fee will also behave differently:

- Concentrated Amount_In B to A -> Rounding Down
- Concentrated Amount_Out B to A -> Rounding Up
- Compounding Amount_In B to A -> Rounding Down
- Compounding Amount_Out B to A -> Rounding Down

Recommendations:

We recommend either adapoting the compounding path to match the concentrated price impact, or clearly document the difference in effect on fees.

Meteora: Resolved with PR-194.

Zenith: Verified.

4.2 Informational

A total of 2 informational findings were identified.

[I-1] `get_next_sqrt_price_from_output` is documented incorrectly

SEVERITY: Informational	IMPACT: Informational
STATUS: Resolved	LIKELIHOOD: Low

Target

- programs/cp-amm/src/liquidity_handler/concentrated_liquidity.rs#L391-L408

Description:

The concentrated liquidity swap math module computes the next sqrt price during swap calculations. This helper is expected to either return a valid next price or fail when documented conditions are not met.

The function comment says it throws when the next price is out of bounds, but the function body does not check `sqrt_min_price` or `sqrt_max_price`. It only asserts non-zero inputs and then calls math helpers.

```
/// Gets the next sqrt price given an output amount of token_a or token_b
/// Throws if price or liquidity are 0, or if the next price is out of bounds
pub fn get_next_sqrt_price_from_output(
    sqrt_price: u128,
    liquidity: u128,
    amount_out: u64,
    a_for_b: bool,
) -> Result<u128> {
    assert!(sqrt_price > 0);
    assert!(liquidity > 0);

    // round to make sure that we don't pass the target price
    if a_for_b {
        get_next_sqrt_price_from_amount_out_b_rounding_down(sqrt_price,
    liquidity, amount_out)
    } else {
```

```
        get_next_sqrt_price_from_amount_out_a_rounding_up(sqrt_price,
    liquidity, amount_out)
    }
}
```

This happens because bound validation is implemented in higher-level caller functions, not in this helper. The helper-level comment was not kept in sync with the execution path.

```
fn calculate_a_to_b_from_amount_out(&self, amount_out: u64) →
    Result<SwapAmountFromOutput> {
    let next_sqrt_price =
        get_next_sqrt_price_from_output(self.sqrt_price, self.liquidity,
    amount_out, true)?;

    if next_sqrt_price < self.sqrt_min_price {
        return Err(PoolError::PriceRangeViolation.into());
    }
    // ...
}

fn calculate_b_to_a_from_amount_out(&self, amount_out: u64) →
    Result<SwapAmountFromOutput> {
    let next_sqrt_price =
        get_next_sqrt_price_from_output(self.sqrt_price, self.liquidity,
    amount_out, false)?;

    if next_sqrt_price > self.sqrt_max_price {
        return Err(PoolError::PriceRangeViolation.into());
    }
    // ...
}
```

As a result, the helper documentation is incorrect about where out-of-bounds checks are enforced.

Recommendations:

We recommend updating the helper comment to state that it validates only non-zero inputs and arithmetic constraints, while price-range checks are enforced by callers. If the intended behavior is helper-level bounds enforcement, add explicit min/max bound checks in this function.

Meteora: Resolved with PR-193.

Zenith: Verified.

[I-2] Inconsisten behavior in case of 0 swaps

SEVERITY: Informational	IMPACT: Informational
STATUS: Resolved	LIKELIHOOD: Low

Target

- programs/cp-amm/src/liquidity_handler/concentrated_liquidity.rs#L373-L407
- programs/cp-amm/src/liquidity_handler/concentrated_liquidity.rs#L438-L445
- programs/cp-amm/src/liquidity_handler/concentrated_liquidity.rs#L457-L464
- programs/cp-amm/src/liquidity_handler/concentrated_liquidity.rs#L490-L500
- programs/cp-amm/src/liquidity_handler/concentrated_liquidity.rs#L504-L514

Description:

In `programs/cp-amm/src/liquidity_handler/concentrated_liquidity.rs`, the functions `get_next_sqrt_price_from_input` and `get_next_sqrt_price_from_output` aree called by token-A or token-B helper functions to compute the next sqrt price. These functions assert non-zero price and liquidity, then route based on `a_for_b`.

```
pub fn get_next_sqrt_price_from_input(
    sqrt_price: u128,
    liquidity: u128,
    amount_in: u64,
    a_for_b: bool,
) → Result<u128> {
    assert!(sqrt_price > 0);
    assert!(liquidity > 0);
    // ...
    if a_for_b {
        get_next_sqrt_price_from_amount_in_a_rounding_up(sqrt_price,
    liquidity, amount_in)
    } else {
        get_next_sqrt_price_from_amount_in_b_rounding_down(sqrt_price,
    liquidity, amount_in)
    }
}
```

```
pub fn get_next_sqrt_price_from_output(
    sqrt_price: u128,
    liquidity: u128,
    amount_out: u64,
    a_for_b: bool,
) → Result<u128> {
    assert!(sqrt_price > 0);
    assert!(liquidity > 0);
    // ...
    if a_for_b {
        get_next_sqrt_price_from_amount_out_b_rounding_down(sqrt_price,
    liquidity, amount_out)
    } else {
        get_next_sqrt_price_from_amount_out_a_rounding_up(sqrt_price,
    liquidity, amount_out)
    }
}
```

What goes wrong is inconsistent zero-amount handling. `get_next_sqrt_price_from_amount_in_a_rounding_up` and `get_next_sqrt_price_from_amount_out_a_rounding_up` explicitly return early when `amount == 0`, but `get_next_sqrt_price_from_amount_in_b_rounding_down` and `get_next_sqrt_price_from_amount_out_b_rounding_down` do not.

```
pub fn get_next_sqrt_price_from_amount_in_a_rounding_up(
    sqrt_price: u128,
    liquidity: u128,
    amount: u64,
) → Result<u128> {
    if amount == 0 {
        return Ok(sqrt_price);
    }
    // ...
}

pub fn get_next_sqrt_price_from_amount_out_a_rounding_up(
    // ...
) → Result<u128> {
    if amount == 0 {
        return Ok(sqrt_price);
    }
    // ...
}
```

```
pub fn get_next_sqrt_price_from_amount_in_b_rounding_down(
    sqrt_price: u128,
    liquidity: u128,
    amount: u64,
) → Result<u128> {
    // no explicit amount == 0 early return
    let quotient = U256::from(amount)
        .safe_shl(128)?
        .safe_div(U256::from(liquidity))?;
    // ...
}

pub fn get_next_sqrt_price_from_amount_out_b_rounding_down(
    // ...
) → Result<u128> {
    // no explicit amount == 0 early return
    let quotient = U256::from(amount)
        .safe_shl(128)?
        .div_ceil(U256::from(liquidity));
    // ...
}
```

This happens because the zero check is implemented in specific helper functions instead of being centralized in `get_next_sqrt_price_from_input` and `get_next_sqrt_price_from_output`.

As a result, zero-amount handling is inconsistent across equivalent price-update paths.

Recommendations:

We recommend adding centralized `amount == 0` checks in `get_next_sqrt_price_from_input` and `get_next_sqrt_price_from_output` in `programs/cp-amm/src/liquidity_handler/concentrated_liquidity.rs`. This keeps zero-amount behavior uniform and lets helper functions focus on non-zero math.

```
// Fixed in concentrated_liquidity.rs: centralized zero check
pub fn get_next_sqrt_price_from_input(
    sqrt_price: u128,
    liquidity: u128,
    amount_in: u64,
    a_for_b: bool,
) → Result<u128> {
    assert!(sqrt_price > 0);
    assert!(liquidity > 0);
    if amount_in == 0 {
```

```
        return Ok(sqrt_price);
    }
    // ...
}

// Fixed in concentrated_liquidity.rs: centralized zero check
pub fn get_next_sqrt_price_from_output(
    sqrt_price: u128,
    liquidity: u128,
    amount_out: u64,
    a_for_b: bool,
) → Result<u128> {
    assert!(sqrt_price > 0);
    assert!(liquidity > 0);
    if amount_out == 0 {
        return Ok(sqrt_price);
    }
    // ...
}
```

```
// optional cleanup: helper keeps non-zero math only
pub fn get_next_sqrt_price_from_amount_in_a_rounding_up(
    // ...
) → Result<u128> {
    if a_for_b {
        get_next_sqrt_price_from_amount_in_a_rounding_up(sqrt_price,
    liquidity, amount_in)
    } else {
        get_next_sqrt_price_from_amount_in_b_rounding_down(sqrt_price,
    liquidity, amount_in)
    }
}
    // removed duplicated `if amount == 0` early return
    // ...
}
```

Meteora: Resolved with PR-192.

Zenith: Verified.